

UNIDAD 6 · SEMANAS 8–9

Señales y memoria compartida

📅 Semanas 8–9 · fechas por confirmar </> Laboratorio TD5 (resuelto + desafío IA)

📺 3 animaciones

MANUAL M6 · TEORÍA DE LA UNIDAD

Señales y memoria compartida

Lectura base de la unidad 6 (≈ 1.5 h). Las salidas son reales: programas del TD5 ejecutados en el equipo del curso.

CONTENIDO

- | | | | |
|----|---|----|---|
| §1 | Señales: envío, pendientes y entrega | §3 | <code>mmap</code> : proyectar archivos en memoria |
| §2 | Manejadores y sincronización sin carreras | §4 | Memoria compartida y su precio |

§1 Señales: envío, pendientes y entrega

Una **señal** es un aviso asíncrono a un proceso. La envía el núcleo (SIGSEGV por un acceso inválido, SIGPIPE por escribir en una tubería sin lector, SIGCHLD cuando termina un hijo), la terminal (Ctrl-C → SIGINT, Ctrl-\ → SIGQUIT, Ctrl-Z → SIGTSTP) u otro proceso con `kill(pid, sig)`. Por cada señal, el proceso tiene una **acción**: la de omisión (terminar, terminar con core, detener, continuar o ignorar), ignorarla (`SIG_IGN`) o ejecutar una función **manejadora**. `SIGKILL` y `SIGSTOP` son las únicas que no se pueden capturar, ignorar ni bloquear.

`kill` solo marca la señal como **pendiente**. El núcleo la **entrega** cuando el proceso vuelve a modo usuario —al terminar una llamada al sistema o al recibir la CPU tras un cambio de contexto—, salvo que esté **bloqueada** por la máscara del proceso: entonces espera. Para las señales estándar, "pendiente" es un bit: si llegan tres iguales mientras está bloqueada, al desbloquear se entrega una sola. Las señales de tiempo real (`SIGRTMIN` ...) sí se encolan y pueden llevar un dato.

Practica esto → A6.1 entrega de señales

§2 Manejadores y sincronización sin carreras

Los manejadores se instalan con `sigaction` (`signal()` tiene semánticas distintas según el sistema). Mientras corre el manejador, la misma señal queda bloqueada (más las de `sa_mask`); con `SA_RESTART`, una llamada al sistema interrumpida se reanuda sola, y si no, falla con `EINTR` (unidad 2). Un manejador interrumpe al programa en cualquier punto, así que solo puede usar

funciones *async-signal-safe* (`write` sí; `printf` y `malloc` no) y comunicarse con el programa mediante variables `volatile sig_atomic_t`.

Esperar una señal con `pause()` tiene una **carrera**: si la señal llega entre la última comprobación y la llamada a `pause`, el manejador corre antes de tiempo y `pause` espera para siempre. La solución es **bloquear** la señal con `sigprocmask`, comprobar la condición y esperar con `sigsuspend(mask)`, que reemplaza la máscara y duerme **en una sola operación atómica**: la señal que llegó "antes" quedó pendiente y se entrega dentro de `sigsuspend`.

C tds/td5/mysignal.c

```
/*
 * TD5 - exercise 1: catch SIGUSR1 with sigaction(2).
 * ./mysignal [n]      handles SIGUSR1 n times (default 3), then restores the default
 * action
 * From another terminal: kill -USR1 <pid>, kill -STOP <pid>, kill -CONT <pid>, kill
 * -KILL <pid>
 * After the n-th SIGUSR1 the default action is back: the next SIGUSR1 terminates the
 * process.
 */
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

static volatile sig_atomic_t received = 0;

static void on_usr1(int sig)
{
    /* Only async-signal-safe functions here: no printf (it may be interrupted mid-
    buffer). */
    static const char msg[] = "handler: received signal ";
    char digits[4] = { (char)('0' + sig / 10), (char)('0' + sig % 10), '\n', 0 };
    if (write_all(STDERR_FILENO, msg, sizeof msg - 1) == -1 || write_all(STDERR_FILENO,
    digits, 3) == -1)
        return;
    received++;
}

int main(int argc, char *argv[])
{
    int max = argc > 1 ? atoi(argv[1]) : 3;
    struct sigaction sa = { 0 }, old;
    sigset_t block, wait_mask;
```

```
printf("pid %d: waiting for SIGUSR1 (%d times)\n", getpid(), max);
fflush(stdout);

sa.sa_handler = on_usr1;
sigemptyset(&sa.sa_mask);           /* nothing extra blocked while the handler runs
*/
sa.sa_flags = 0;
exit_if(sigaction(SIGUSR1, &sa, &old) == -1, "sigaction");

/* Block SIGUSR1 outside sigsuspend: a signal arriving between the test of
`received`
and the wait stays pending instead of being lost (the pause() race). */
sigemptyset(&bblock);
sigaddset(&bblock, SIGUSR1);
exit_if(sigprocmask(SIG_BLOCK, &bblock, &wait_mask) == -1, "sigprocmask");
sigdelset(&wait_mask, SIGUSR1);

while (received < max)
    sigsuspend(&wait_mask);           /* atomically: unblock SIGUSR1 and sleep */

exit_if(sigaction(SIGUSR1, &old, NULL) == -1, "sigaction restore");
printf("default action restored: the next SIGUSR1 terminates me\n");
fflush(stdout);
exit_if(sigprocmask(SIG_UNBLOCK, &bblock, NULL) == -1, "sigprocmask");
for (;;)
    pause();
}
```

Salida real

```
$ ./mysignal 2 & P=$!; sleep 0.3; kill -USR1 $P; sleep 0.2; kill -STOP $P; kill -USR1
$P; sleep 0.2; kill -CONT $P; sleep 0.2; kill -USR1 $P; wait $P; echo "exit status: $?
(128 + 10 = killed by SIGUSR1)"
pid 357338: waiting for SIGUSR1 (2 times)
handler: received signal 10
handler: received signal 10
default action restored: the next SIGUSR1 terminates me
bash: line 1: 357338 User defined signal 1      ./mysignal 2
exit status: 138 (128 + 10 = killed by SIGUSR1)
```

Otras herramientas: `alarm(s)` programa un SIGALRM; `sleep` en glibc usa `clock_nanosleep` (no `alarm`); `signalfd` convierte las señales en datos que se leen de un descriptor, y `pidfd_send_signal` evita enviar la señal a un PID ya reutilizado.

Practica esto → TD5 · 1 y 2

§3 mmap : proyectar archivos en memoria

`mmap(addr, len, prot, flags, fd, offset)` reserva un rango de direcciones y lo asocia a un archivo (o a nada, con `MAP_ANONYMOUS`). No copia nada: cada página se trae al primer acceso con un fallo de página (unidad 4). Con `MAP_SHARED` las escrituras van a la caché de páginas del archivo, las ven los demás procesos que lo mapean y llegan al disco (`msync` o al desmapear); con `MAP_PRIVATE` la primera escritura copia la página y el archivo no cambia. `len` se redondea a páginas; `offset` debe ser múltiplo del tamaño de página.

Acceder más allá del final del archivo dentro de la última página devuelve ceros que no se guardan; tocar una página completa sin archivo detrás produce **SIGBUS** (la dirección es válida, pero no hay datos que traer). Para agrandar un archivo mapeado hay que usar `ftruncate` antes. Las protecciones `PROT_*` se aplican por página; en x86, escribir implica poder leer:

Salida real

```
$ ./mmap_limits
file size 128 bytes, page size 4096 bytes, mapping of 8192 bytes
wrote bytes 0..4095; byte 4096 raised SIGBUS
file size after the writes: 128 bytes
```

Salida real

```
$ ./map_write
PROT = PROT_WRITE
pointer: 0x7c7095214000
read:   *p = 0
write:  *p = 42
```

Salida real

```
$ ./map_exec
PROT = PROT_EXEC
pointer: 0x75b3a49d8000
bash: line 1: 357495 Segmentation fault      (core dumped) ./map_exec
[exit status 139]
```

Practica esto → A6.3 los límites de mmap · TD5 · 3 y 4

§4 Memoria compartida y su precio

Compartir memoria es hacer que páginas virtuales de dos procesos apunten al mismo marco físico. Hacen falta un **punto de encuentro** y una interfaz: `mmap` de un archivo común con `MAP_SHARED`; POSIX `shm_open` + `mmap` (un nombre en `/dev/shm`, recomendado hoy); o System V: `ftok(archivo, id)` genera una clave, `shmget` crea o encuentra el segmento, `shmat` lo adjunta, `shmdt` lo separa y `shmctl(IPC_RMID)` lo destruye. El segmento sobrevive a los procesos hasta que alguien lo destruye:

Salida real

```
$ ./shm_counter; ./shm_counter; ./shm_counter; ./shm_counter --remove
segment 65593 (key 0x4605476d): counter = 1
segment 65593 (key 0x4605476d): counter = 2
segment 65593 (key 0x4605476d): counter = 3
segment 65593 removed
```

Dos reglas. Primera: cada proceso ve la zona en una **dirección distinta**, así que no se guardan punteros en ella (se guardan desplazamientos). Segunda: la memoria compartida es la comunicación más rápida porque no pasa por el núcleo, pero por eso mismo **nada la sincroniza**. Si varios procesos hacen "leer, sumar 1, escribir" sobre el mismo contador, se pierden actualizaciones; con una operación atómica del procesador (`__atomic_add_fetch`, `lock_xadd` en x86) no:

Salida real

```
$ for i in 1 2 3 4 5; do ./shm_race 5 | cut -d: -f1; done
19 processes registered (expected 32)
28 processes registered (expected 32)
25 processes registered (expected 32)
23 processes registered (expected 32)
27 processes registered (expected 32)
```

Salida real

```
$ ./shm_fork
16 processes registered (expected 16): 357564 357566 357567 357572 357568 357571 357569
357565 357574 357573 357570 357576 357578 357575 357577 357579
```

Practica esto → A6.2 memoria compartida · TD5 · 5